

Exposé: Aufgabenverteilung im WASP-Projekt

Ivan Kurilin

Dimitrij Pivovar

11. Dezember 2024

Projekt-Ziel

Das Ziel in diesem WASP-Projekt ist es, ein Pool-Billard in einer 3D-Umgebung zu implementieren. Die Objekte sollten eine nahezu realistische Reaktion auf Kollisionen mit dem Cue, den Banden, den Taschen und den Kugeln darstellen.

Aufgabenverteilung

Ivan Kurilin

- **[AP] Cue mit Kugelkollision:** Interaktion zwischen dem Cue und der Kugel, wie sie in der Simulation behandelt wird. Dabei kann der User mit einem Maus-Klick den Cue verwenden, interagieren, den Cue ziehen und die Kraft auf die Kugel leiten.
- **[AP] Kollision mit Banden:** Kollisionseffekte, die zwischen den Kugeln und den Banden auftreten.
- **[AP] Billard-Aufstellung:** Objekte auf dem Billardtisch zu Beginn des Spiels, um die Simulation zu starten.
- **[AP] Zentraler vollkommen unelastischer Stoß:** Hiermit möchten wir eine möglichst realistische Interaktion zwischen dem Cue und der Kugel darstellen.

Dimitrij Pivovar

- **[AP] 3D-Kugeln:** Modellierung der 3D-Kugeln, einschließlich ihrer Bewegungen und Interaktionen auf dem Tisch.
- **[AP] 3D-Szene:** Die Szene ist aus verschiedenen Blickwinkeln betrachtbar.
- **[AP] 2D-Tisch-Erweiterung (Billardtaschen, Banden):** Erweiterung des 2D-Tisches, die Billardtaschen und Banden integriert, um eine realistische Interaktion der Kugeln zu ermöglichen.

Mögliche Erweiterung (wird im Team aufgeteilt)

- **[AP] Kugeldesign:** Das Design der Kugeln, einschließlich Farben und Zahlen, die im Billard verwendet werden.
- **[AP] Regelsystem & Punktesystem:** Die Definition der Regeln, die das Spiel steuern, einschließlich der Bedingungen für Gewinnen und Verlieren.

Unterschied zum WPF

- 3D-Szene, unterschiedliche Kollisionen gleichzeitig (elastische/unelastische Kollisionen), Interaktion mit der Umgebung (man kann den Cue nutzen, Punkte erzielen).

Zeitaufwand

- Geschätzte Deadline: 10.03.2025.
- Geschätzter Zeitaufwand: 9 Arbeitspakete mit jeweils 1–2 Wochen pro [AP]. Damit kommen wir auf ca. 756 Stunden, auf 2 Teilnehmer aufgeteilt.

Erwartete Ergebnisse

- Ein funktionierender 3D-Billard-Demonstrator.
- Unterstützung von Kollisionserkennung für verschiedene Objekte.
- Ein dokumentierter Bericht über die Umsetzung und die Herausforderungen.

1 3D Scene

- Besteht aus einer Box, die als Tischoberfläche dient mit den Maßen $x=300$, $y=30$, $z=200$
- Beine mit je Maßen $x=50$, $y=100$, $z=40$. Mit 2 Schleifen implementiert:
 - Äußere Schleife beginnt bei den Koordinaten $x = -90$, Kondition ist das $x \neq 90$ ist und das Inkrement ist 180. Das sorgt dafür das die innere Schleife auf die linken (vom Startpunkt) Beine angewandt wird bevor die x Koordinate um 180 erhöht wird und dann die rechten Beine gezeichnet wird
 - Innere Schleife beginnt bei den Koordinaten $x = -90$, $z = -40$. z wird nach dem Schleifenkörper um 80 erhöht um auf die Position des hinteren Beins zu wechseln.
 - Zusammenfassend kann man sagen die Beine haben folgende Koordinaten:
 - * Bein links hinten $x = \text{width}/2 - 90$, $y = \text{height}/2+55$, $z = -40$
 - * Bein links vorne $x = \text{width}/2 - 90$, $y = \text{height}/2+55$, $z = 40$
 - * Bein rechts hinten $x = \text{width}/2 + 90$, $y = \text{height}/2+55$, $z = -40$
 - * Bein rechts vorne $x = \text{width}/2 + 90$, $y = \text{height}/2+55$, $z = 40$
- TODOs
 - Bounding Box für Kugeln auf Tischoberfläche begrenzen
 - Kameraposition verbessern für Startszene
 - Kugeln in der Pyramide aufstellen für Startszene
 - Taschen implementieren
 - Kollision zwischen der BillardCue und Kugeln verbessern
 - Quellen + Formeln richtig Zitieren
- Herausforderungen
 - Allgemeines Verständnis der `translate()` Funktion
 - * Erstes Learning war, dass die `translate` Funktion akkumuliert. `translate(50,0,0)` und danach `translate(70,0,0)` summiert sich auf. `pushMatrix()`, `popMatrix()` hilft

2 Kollisionsdetektion und -antwort

Die Kollisionsdetektion zwischen den Kugeln erfolgt, indem der Abstand zwischen den Mittelpunkten der beiden Kugeln berechnet wird. Wenn dieser Abstand kleiner ist als die Summe der Radien der Kugel, wird eine Kollision erkannt. Die Berechnung des Abstands erfolgt mit der Formel:

$$\text{Abstand} = \sqrt{(dx)^2 + (dy)^2}$$

Dabei sind dx und dy die Differenzen der x - und y -Koordinaten der beiden Kugeln.

2.1 Zentraler elastischer Stoß

Einen Stoß kann man als elastisch bezeichnen, falls sich die Energie der Objekte nach dem Stoß erhalten bleibt. Wir können den elastischen Stoß durch folgende Gleichung virtuell darstellen:

$$m_1v_1 + m_2v_2 = m_1v'_1 + m_2v'_2$$

- m_1, m_2 : die Massen der beiden Objekte
- v_1, v_2 : die Geschwindigkeiten der beiden Objekte

Die rechte Seite der Gleichung steht für den Impuls nach dem Stoß.

2.2 Normalvektor und Impulsberechnung

Der Normalvektor (nx, ny) definiert die Richtung der Kollision. Er wird durch die Berechnung des Abstands zwischen der Kugel bestimmt.

2.3 Placeholder